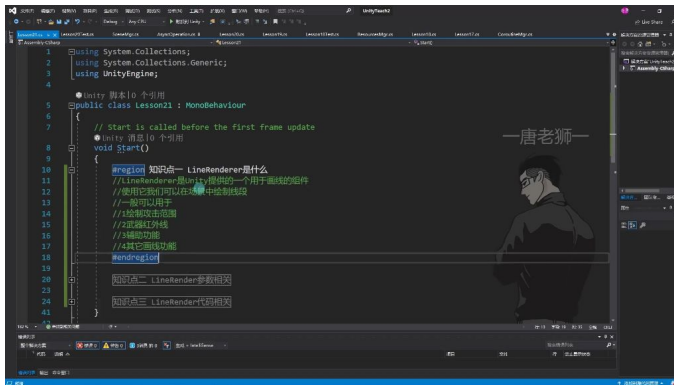


一、unity基础画线功能 00:04

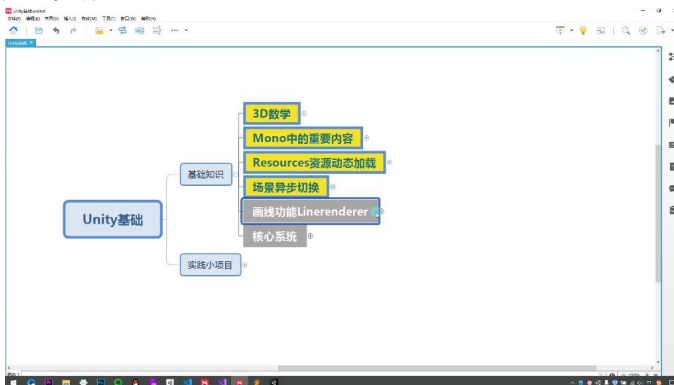
1. 知识点一、LineRenderer是什么 00:29



-
- **组件功能:** Unity提供的专门用于画线的组件，可在场景中绘制线段
- **主要用途:**
 - 绘制攻击范围
 - 武器红外线
 - 辅助功能
 - 其他画线需求（如圆形、方形或不规则线段）
- **本质理解:** 通过设置多个点坐标，将这些点连接起来形成线段

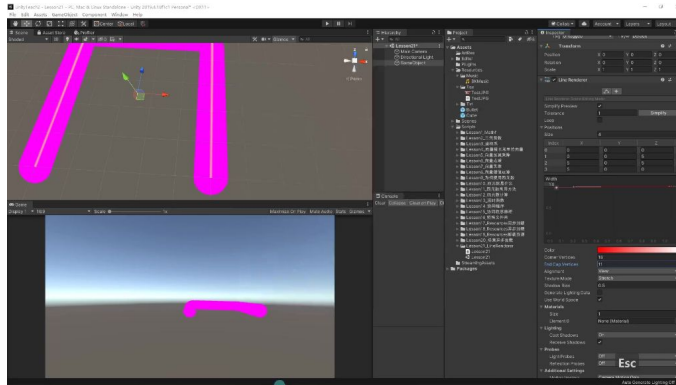
2. 知识点二、LineRender参数相关 01:04

1) 核心参数



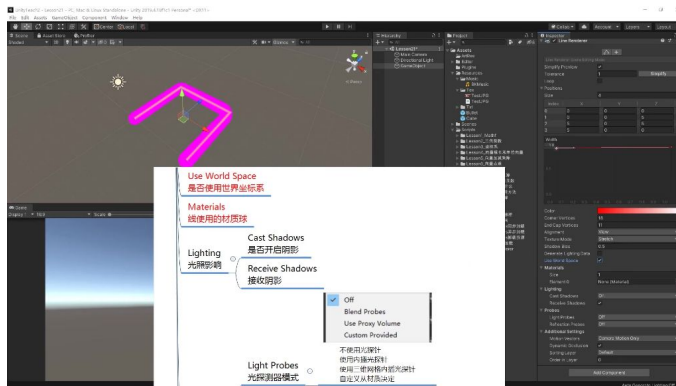
-
- **Loop:**
 - 作用：控制线段首尾是否自动相连
 - 效果：勾选后会将终点和起点自动连接形成闭合图形
- **Positions:**
 - 作用：设置构成线段的各个点坐标
 - 特点：默认使用世界坐标系（除非取消勾选Use World Space）
 - 示例：两点构成直线，多个点可构成复杂图形
- **Width:**
 - 作用：调整线段宽度曲线
 - 功能：可设置开始和结束宽度，添加曲线实现渐变效果
 - 注意：通常保持默认，特殊需求才调整
- **Color:**
 - 作用：设置线段颜色渐变
 - 特点：可调整透明度和颜色过渡效果
 - 注意：需要配合材质才能显示效果

2) 圆角参数



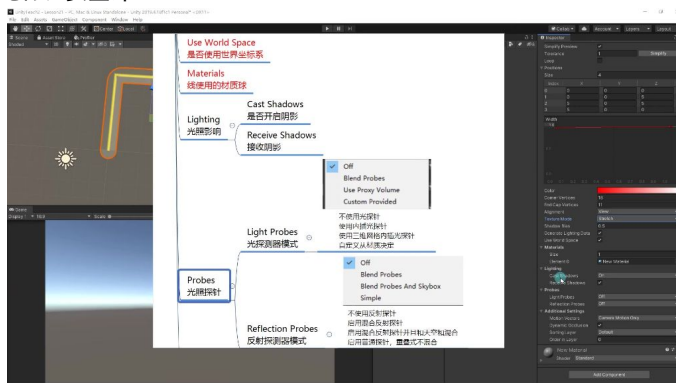
- **Corner Vertices:**
 - 作用：控制线段转角处的圆滑度
 - 原理：通过增加额外顶点使转角更平滑
 - 建议值：3-5即可达到较好效果，过高影响性能
- **End Cap Vertices:**
 - 作用：控制线段终点的圆角效果
 - 区别：与Corner Vertices分别控制不同位置的圆角

3) 坐标系与材质



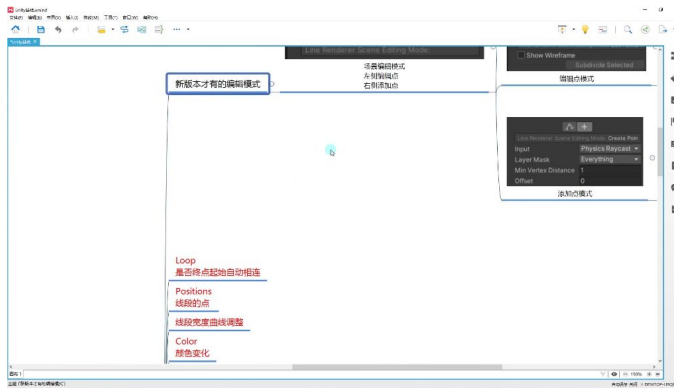
- **Use World Space:**
 - 关键作用：决定线段是否随物体移动
 - 效果：取消勾选后线段会跟随物体坐标系移动
- **Materials:**
 - 作用：设置线段使用的材质球
 - 注意事项：
 - 需要创建专门材质
 - 受光材质需开启"Generate Lighting Data"
 - 贴图模式影响材质显示效果

4) 光照与渲染



- **Lighting:**
 - Cast Shadows: 控制是否投射阴影
 - Receive Shadows: 控制是否接收阴影
- **Probes:**
 - 光照探针: 影响光照效果的高级设置
 - 反射探针: 控制反射效果的模式
 - 注意: 这些参数通常保持默认即可

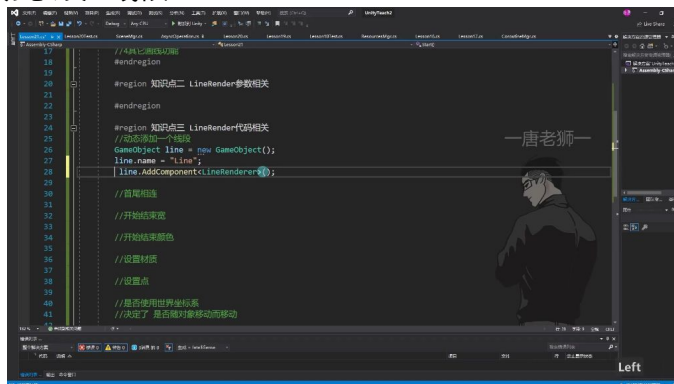
5) 新版本编辑功能



- **Simplify Preview:**
 - 作用: 显示黄色辅助线便于观察
- **编辑模式:**
 - 左侧编辑点: 可调整现有点位置
 - 右侧添加点: 支持多种添加方式
 - 物理射线检测 (最常用)
 - 鼠标位置检测 (较少使用)
 - 最小顶点距离: 控制连续绘制时的点密度
- **使用建议:** 代码绘制更常用, 此功能适合特定场景

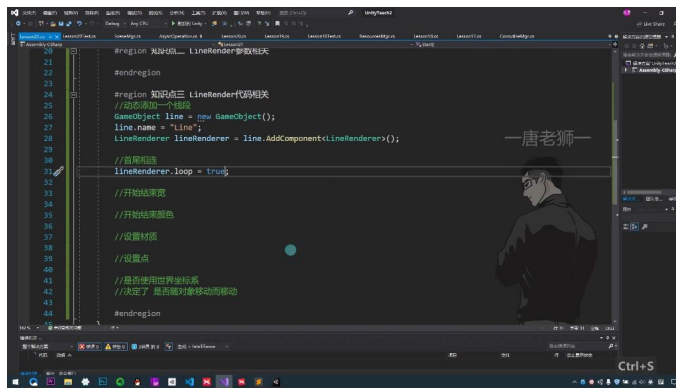
3. 知识点三、LineRenderer代码相关 05:25

1) 动态添加线段 25:36

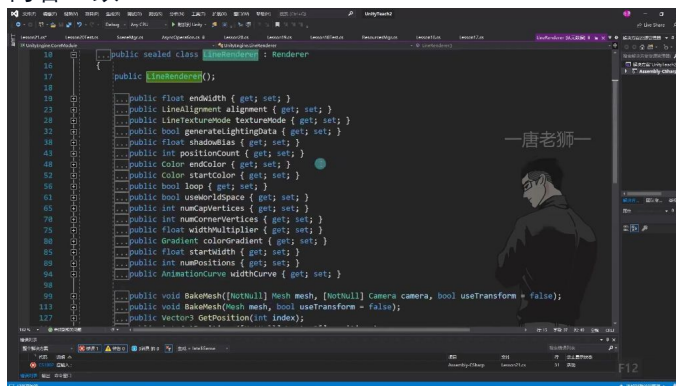


- **创建方法:** 通过代码动态创建线段对象, 无需预先在场景中放置物体
- **实现步骤:**
 - 使用GameObject line = new GameObject()创建空物体
 - 通过line.name = "Line"设置对象名称
 - 添加LineRenderer组件: `LineRenderer lineRenderer = line.AddComponent<LineRenderer>()`
- **组件特性:** LineRenderer是Unity引擎内置组件, 属于UnityEngine命名空间

2) 首尾相连 27:16

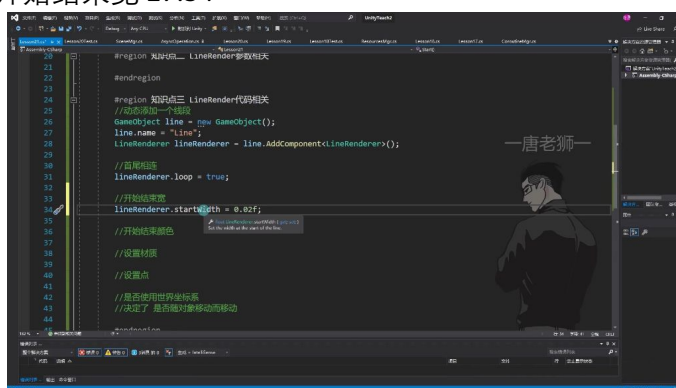


-
- **关键参数：**通过lineRenderer.loop = true实现线段首尾自动连接
- **参数说明：**
 - true：起点和终点自动连接形成闭合图形
 - false：保持线段开放状态（默认值）
- **参数查看方法：**可通过F12查看LineRenderer类的所有可用参数，与Inspector面板显示内容一致

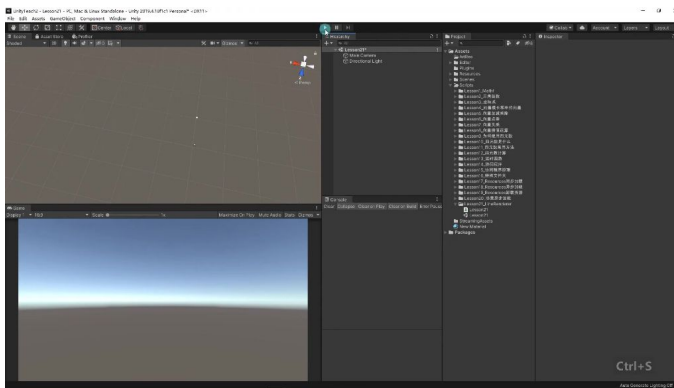


-
- **其他重要参数：**
 - startWidth/endTime：控制线段起止宽度
 - startColor/endTime：设置线段起止颜色
 - useWorldSpace：决定线段是否随对象移动而移动

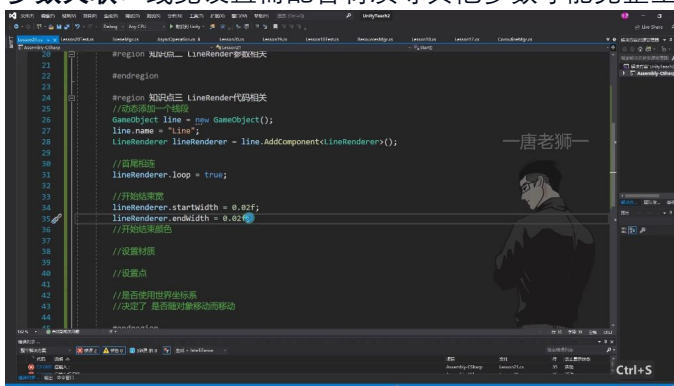
3) 开始结束宽 27:54



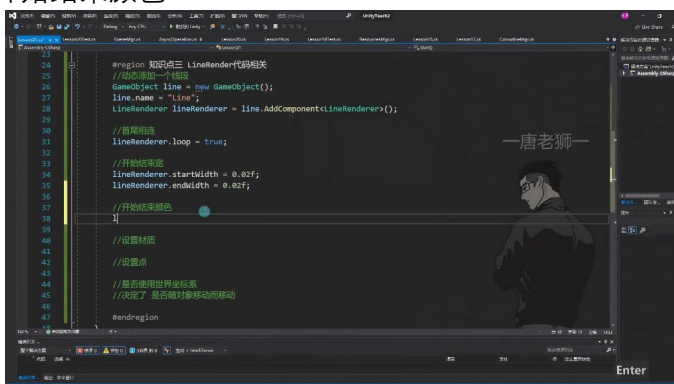
-
- **设置方法：**通过代码lineRenderer.startWidth和lineRenderer.endTime分别设置线段开始和结束的宽度
- **取值范围：**一般设置在0到1之间，0.02f已经是非常细的线宽
- **同步设置：**建议将开始和结束宽度设为相同值（如0.02f），这样线段从头到尾宽度一致
- **视觉效果：**当线宽设置过小时，在场景中可能难以直观观察到宽度差异



-
- **调试技巧：**运行代码后可在场景视图中观察线段实际渲染效果
- **常见问题：**若未正确设置线宽可能导致线段显示异常或不可见
- **参数关联：**线宽设置需配合材质等其他参数才能完整呈现线段效果



- - **代码规范：**
 - **命名建议：**创建LineRenderer对象时应赋予明确名称（如"Line"）便于后续管理
 - **组件添加：**通过AddComponent<LineRenderer>()方法动态添加线段渲染组件
- 4) 开始结束颜色 29:13



-
- **颜色渐变特性：**LineRenderer的颜色支持从开始到结束的渐变效果，可以通过代码控制起始和结束颜色
- **Color结构体：**Unity提供了内置的Color结构体，包含预定义颜色和RGBA通道设置



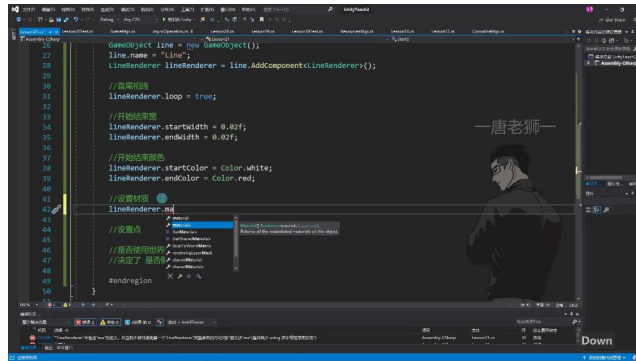
○ R: 红

-
- The screenshot displays the Unreal Engine 4 development environment. The top-left pane shows a 3D perspective view of a simple scene with a ground plane and a few basic geometric shapes. The bottom-left pane is the console window, which contains several error messages, including 'Warning: Actor Destroyed: Actor not found' and 'Warning: Actor Destroyed: Actor not found'. The right-hand side of the interface is occupied by the 'Properties' panel, which shows the 'Static Mesh' properties of the selected object, including its name, class, and various settings like 'bUseInstancing' and 'bUseStreaming'. The top bar indicates the current project is 'Unreal Engine 4' and the editor is in 'Edit' mode.

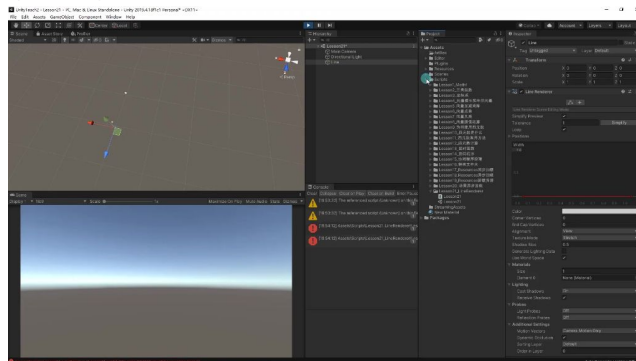


- LineRenderer基础设置

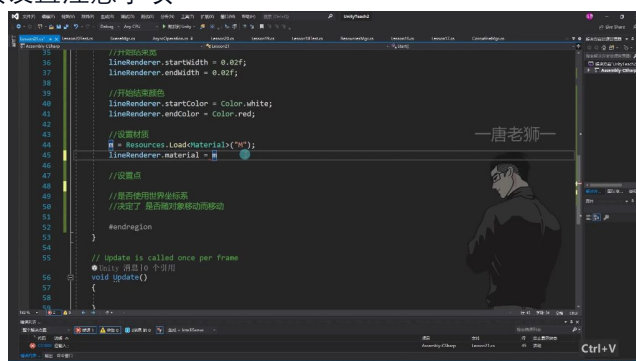
- LineRenderer基础设置



- 创建对象：通过GameObject line = new GameObject()创建新游戏对象
- 命名对象：使用line.name = "Line"为对象命名
- 添加组件：通过AddComponent<LineRenderer>()添加线渲染器组件
- 首尾相连：设置lineRenderer.loop = true可使线条首尾相连
- 宽度设置：使用startWidth和endWidth属性设置线条粗细，如0.02f
- 颜色设置：通过startColor和endColor属性设置线条颜色，支持渐变效果
- 材质动态加载



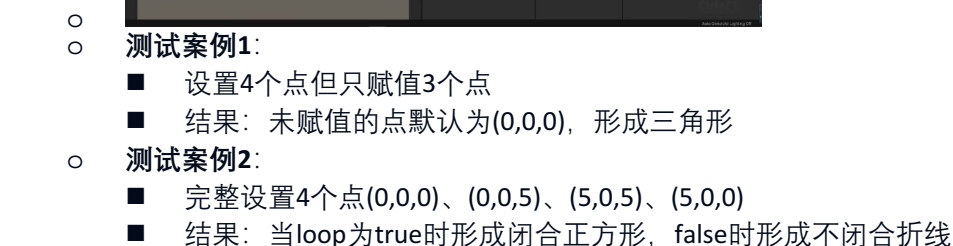
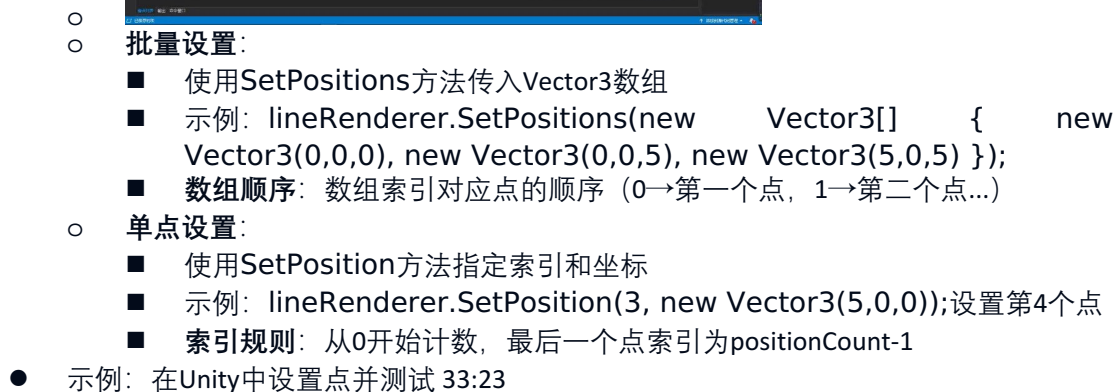
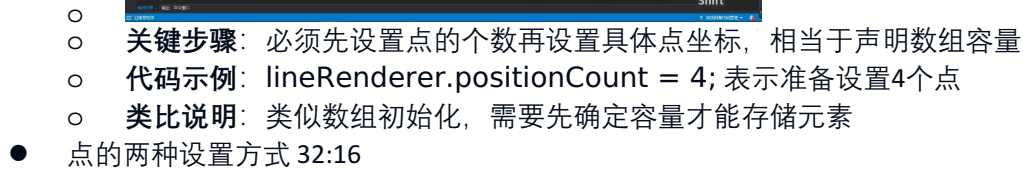
- 资源准备：将材质球放入Resources文件夹，命名为"M"
- 加载方法：使用Resources.Load<Material>("M")动态加载材质
- 赋值材质：将加载的材质赋值给LineRenderer的material属性
- 类型指定：加载时需要明确指定资源类型为Material
- 代码示例：
- 材质设置注意事项



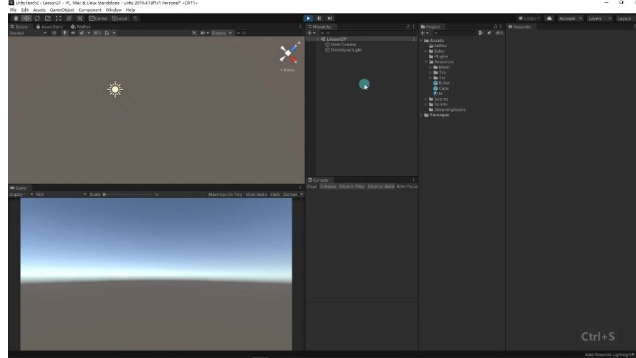
- 资源路径：材质必须放在Resources文件夹下才能被加载
- 类型匹配：加载时要确保类型与实际资源类型一致
- 赋值时机：建议在Start或Awake方法中进行材质加载和赋值
- 效果验证：即使未设置点坐标，也可通过Inspector窗口验证材质是否成功应用
- 调试技巧：可通过打印日志或断点调试确认材质是否加载成功

6) 设置点 31:44

- 设置点个数 31:50

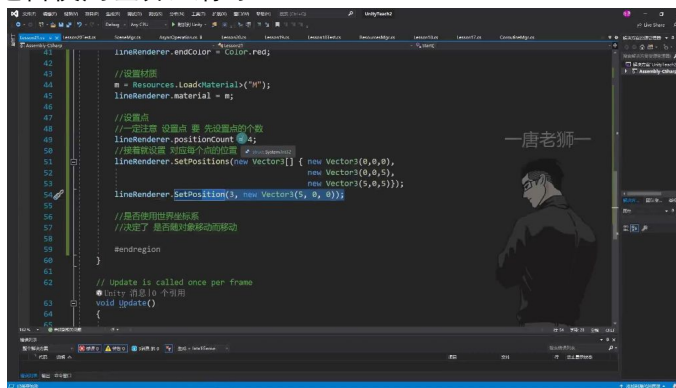


- 注意事项：点数量与赋值 33:36

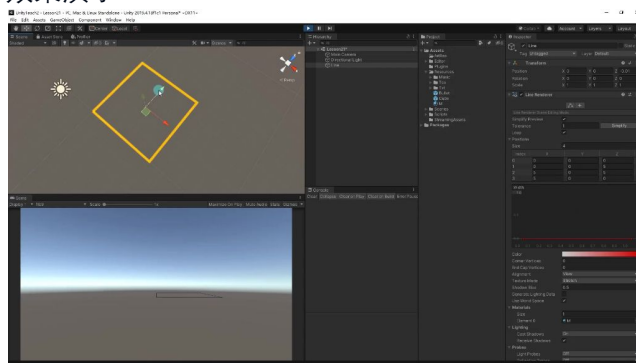


- 必须匹配：设置的positionCount必须与实际赋值的点数一致
- 默认值风险：未赋值的点会自动设为(0,0,0)，可能导致意外连线
- 调试建议：
 - 检查Inspector中positions属性确认各点坐标
 - 修改loop参数验证闭合效果
- 操作顺序：positionCount→SetPositions/SetPosition→检查效果

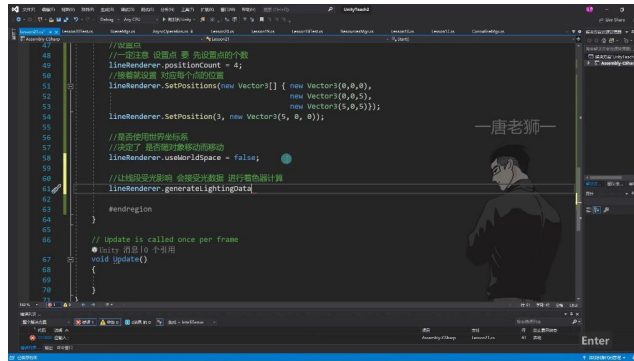
7) 是否使用世界坐标系 35:43



- 基本概念
 - 作用原理：决定LineRenderer绘制的线段是否随对象移动而移动
 - 默认状态：useWorldSpace参数默认勾选（true）
 - 代码设置：通过lineRenderer.useWorldSpace = false取消世界坐标系
- 实际效果演示

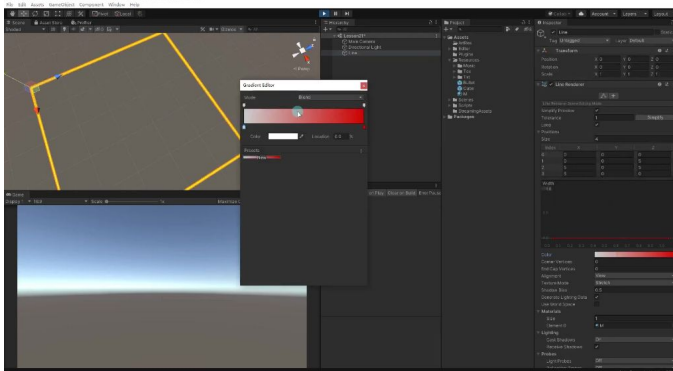


- 勾选状态：
 - 移动对象时线段保持世界坐标不变
 - 表现为线段位置固定，不随对象移动
- 取消状态：
 - 线段会跟随对象移动
 - 需要显式设置useWorldSpace = false
- 光照数据处理



- 黑色线段问题：材质可能未正确接收光照数据
- 解决方法：
 - 设置lineRenderer.generateLightingData = true
 - 使线段能接收光照数据进行着色器计算
- 效果验证：设置后线段会显示正确材质颜色而非黑色
- 使用注意事项
 - 设置顺序：
 - 必须先设置positionCount确定点数
 - 再通过SetPositions设置各点坐标
 - 坐标设置：
 - 材质加载：使用Resources.Load<Material>("M")加载材质资源

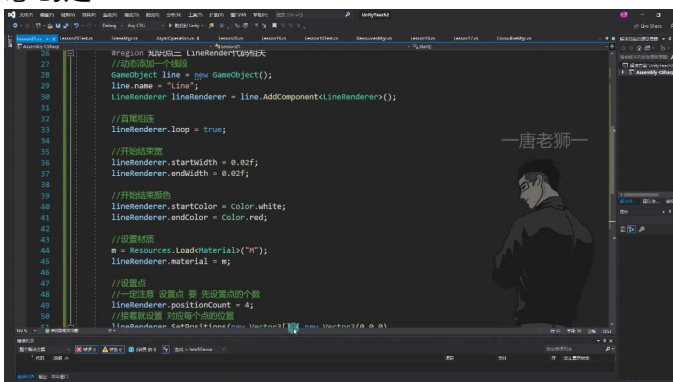
二、LineRenderer的功能 37:28



- 基本功能: Unity提供的划线脚本组件，用于在3D空间中绘制线段
- 材质特性：
 - 某些材质不会受到颜色设置的影响（如演示中设置红色但未生效的情况）
 - 需要特定材质才能正确显示颜色效果

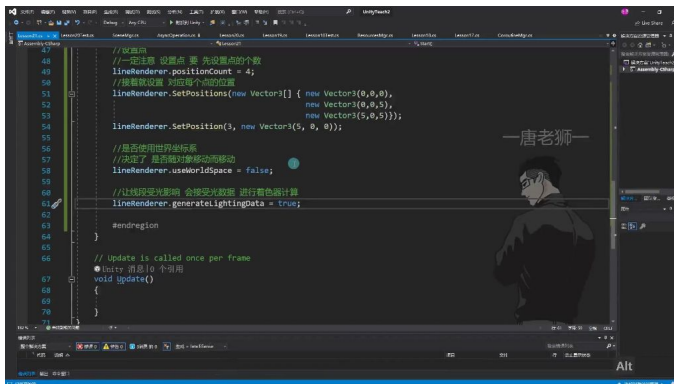
三、通过代码控制LineRenderer 37:50

1. 动态创建LineRenderer



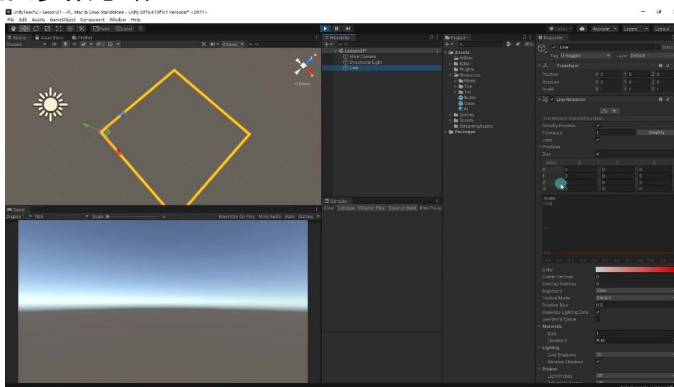
- 创建步骤:
 - i. 创建新GameObject并命名
 - ii. 添加LineRenderer组件
 - iii. 设置基本参数:
 - loop = true 使线段首尾相连
 - startWidth/endWidth = 0.02f 设置线宽
 - startColor/endColor 设置颜色渐变
- 材质加载:
 - 使用Resources.Load<Material>("M")加载材质
 - 通过material属性赋值给LineRenderer

2. 关键参数设置



- 点设置顺序:
 - 必须先设置positionCount确定点数
 - 再通过SetPositions或SetPosition设置具体坐标
- 坐标系选择:
 - useWorldSpace = false使用局部坐标系, 会随对象移动
 - 设置为true则使用世界坐标系
- 光照影响:
 - generateLightingData = true使线段接受光照着色计算

3. 重要参数总结



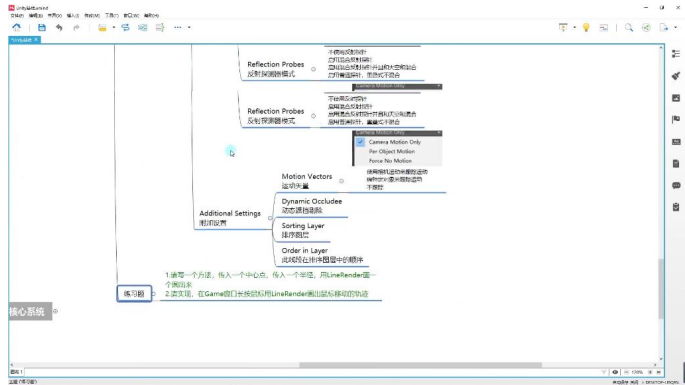
- 核心参数:
 - Positions: 定义线段的各个顶点位置
 - Width: 控制线段粗细
 - Color: 设置颜色渐变效果
 - Loop: 是否首尾相连
- 高级设置:
 - Corner Vertices: 控制转角圆滑度
 - End Cap Vertices: 控制端点圆滑度

- Alignment: 线段对齐方式（面向摄像机或Z轴）
- Texture Mode: 纹理映射方式

四、课程内容总结 38:28

- 三大要点:
 - LineRenderer是Unity提供的专业划线工具
 - 掌握所有重要参数的含义和设置方法
 - 学会通过代码动态控制LineRenderer

1. 练习题



- 编写方法: 根据中心点和半径用LineRenderer画圆
- 实现功能: 在Game窗口用鼠标轨迹画线

2. 重点提醒

- 代码控制要点:
 - 所有参数都可通过代码动态修改
 - 重点掌握位置点设置、颜色、宽度等核心属性
 - 注意设置点的顺序: 先设数量再设位置

五、知识小结

知识点	核心内容	重点/易混淆点	难度系数
LineRenderer 组件作用	Unity提供的专用画线组件，可绘制攻击范围、武器红外线等	与普通Renderer的区别	★★
Loop 参数	控制线段首尾是否自动连接	勾选后自动闭合图形	★★
Positions 参数	定义线段顶点坐标数组	世界坐标系与本地坐标系切换	★★★
Width 曲线	控制线段粗细变化	曲线调整影响整体过渡效果	★★★
Color 渐变	设置线段颜色过渡	需配合材质才能显示效果	★★
Corner Vertices	调整转角圆滑度	值越大转角越平滑	★★

End Vertices	控制端点圆角	独立于转角参数	★★
Use World Space	坐标系切换开关	决定线段是否随父物体移动	★★★
材质设置	需配合贴图使用	受光材质需开启Lighting Data	★★★
动态创建代码	AddComponent<LineRenderer>()	必须先设置顶点数再赋值	★★★
SetPositions 方法	批量设置顶点坐标	未赋值顶点默认为(0,0,0)	★★★
编辑模式功能	新版场景直接编辑工具	物理射线检测添加顶点	★★